

TalentPilot-AI

Replacing spreadsheet self-reporting
with automatic, trustworthy skill-
readiness tracking.

PROBLEM · SOLUTION · REQUIREMENTS · ARCHITECTURE

STATUS

PRD & Architecture — Final

LAUNCH DATE

13 July 2026

BUILD SCOPE

Local pilot, zero budget

DOCUMENT OWNER

TalentPilot

CONTENTS

Inside this document

01	The Problem	Why the spreadsheet-driven process breaks down, and the data behind it
02	The Solution	How automatic video-progress capture replaces manual self-reporting
03	Who It's For	Roles in scope, and what's deliberately excluded
04	Requirements	The 14 functional requirements, grouped by feature, in plain language
05	Architecture	How the system is built, and the reasoning behind the key decisions
06	Open Questions	Risks and gaps carried forward honestly from planning
07	Source Documents	The BMAD planning artifacts this was compiled from

Compiled from the BMAD planning artifacts in `_bmad-output/` — Product Brief, PRD, Architecture Spine, Solution Design, and Epics.

01 The Problem Statement

HR at SAILS Software tracks employee skill development through a shared spreadsheet. HR assigns "must-do" skills, employees are supposed to self-report progress, and HR periodically scans the grid to judge who's ready for what.

This breaks down for one structural reason: **a spreadsheet only works if people remember to update it — and nobody has a reason to.**

Data is unreliable

88% of HR spreadsheets contain errors, and only 20% of HR leaders trust their own skill data.

Visibility is manual

Consolidating status across dozens of employees eats real admin time — there's no live org-wide picture.

No payoff for employees

Self-reporting is a chore with zero personal upside, so it's the first thing skipped.

HR ends up guessing

When asked "who's ready for this?", HR often has to ask around anyway — the sheet can't be trusted at the moment it matters.

"87% of companies cite lack of workforce-skills visibility as their top barrier to growth."

Industry research, cited in the Product Brief

The key insight driving this project: **it isn't the spreadsheet format that's broken — it's the self-reporting step.** Any tool that still asks employees to type in "I'm 40% done" inherits the same trust problem. The only real fix is to remove manual reporting wherever it's technically possible to replace it with something automatic.

02 The Solution

TalentPilot-AI removes the self-reporting step for video-based learning — the one content type where progress can be captured automatically, from actual behavior instead of a manual entry.

Core mechanic: the same signal that lets an employee "resume where they left off" is the signal that tells HR how far along they are. One write, two payoffs, no separate status update.

How it works, end to end

1. **HR assigns a skill** to an employee — this sets the roadmap. The system never guesses what someone should learn.
2. **The system recommends content** (video, document, or website) matched to that skill using AI-based semantic matching — no manual hunting.
3. **The employee watches the video**, and playback position is recorded silently in the background — no button, no form field.
4. **The employee resumes exactly where they left off**, any time — Netflix/Spotify-style.
5. **HR's dashboard updates itself** from that same watch data, in near real time — no re-typing, no chasing.
6. **HR can drill into any row** to see the raw evidence — watch %, last-updated time, and whether it's machine-verified or still self-reported.

Success looks like

- HR retires the shadow spreadsheet within 60 days of go-live.
- Stale, self-reported video-progress rows drop from ~100% today to near-zero.
- Employees finish videos and use resume, without ever being asked to log anything.

Counter-metric to watch: if HR is still privately checking the old spreadsheet after go-live, that's a red flag even if the backend numbers look clean — behavior change is the real proof, not the metric alone.

03 Who This Is For

PRIMARY USER

HR Admin — “Rita”

Assigns skills, reads the dashboard, makes readiness calls, and can manually mark someone ready when she has other grounds for confidence.

SECONDARY USER

Employee — “Casey”

Receives assigned skills, watches recommended content, resumes videos. Never types a status update.

NOT IN V1

Manager / Team Lead

Explicitly not a role in this product. Considered and dropped — revisit only if a real pilot demands it.

Explicitly out of scope — on purpose

- ✗ **Not a full LMS/LXP** — no course catalog, no learning paths, no certifications.
- ✗ **Not a skill-gap detector** — the system never decides what someone should learn; HR always assigns explicitly.
- ✗ **Not a performance-evaluation input** — auto-captured watch data is coaching-only, structurally blocked from performance reviews.
- ✗ **Not tracking documents or websites** — only video can be auto-verified; other content stays visibly self-reported.
- ✗ **No content-approval workflow** — AI-recommended content reaches employees without a human QA gate (accepted risk).
- ✗ **Not a commercial product** — an internal pilot for SAILS Software's own HR function.

04 Project Requirements

The PRD defines 14 functional requirements across five features. Below is what each one means in practice.

PROVENANCE LABELS USED THROUGHOUT:

● Verified

● Self-reported

● Needs Attention

● HR Override

4.1 — Skill Assignment Flow

FR-1 / FR-2

HR assigns a skill, with content shown up front

A three-step flow — pick employee → pick skill → review the AI-suggested content → confirm — completes in under two minutes. HR sees what content the employee will get before confirming (visibility, not an approval gate). A skill can be assigned without content if nothing matches yet. Re-assigning a skill an employee already has surfaces the existing record instead of silently duplicating it. Canceling mid-flow leaves nothing behind.

4.2 — AI-Assisted Content Discovery

FR-3 / FR-4

One relevant recommendation per skill, no search box

For every assigned skill, the system suggests one piece of content using approximate/semantic matching — exact keyword overlap isn't required. If nothing clears the relevance bar, it shows no recommendation rather than a poor guess. Employees see all their assigned content in one simple list grouped "In Progress" / "To Start" — never a browsable catalog, since HR already decided what they need. Content refreshes via a scheduled batch job, because the video-search service has a strict daily quota.

4.3 — Automatic Video Progress Capture & Resume

FR-5 - FR-7

Silent capture, exact resume, correct ordering

Playback position is recorded automatically every 5–10 seconds — no manual input field exists anywhere. A closed tab still reliably saves the last position. Resume lands at the *exact* spot on first use — getting this wrong is treated as launch-blocking, not minor. If the same video plays in two tabs, the system keeps whichever update happened most recently *in time*, never just the higher number — so an intentional rewind is respected instead of blocked.

4.4 — Readiness Dashboard

FR-8 - FR-12

Status at a glance, trust one click away

Each row shows a simple Status badge — *Not Started*, *In Progress*, or *Completed*. Clicking in reveals the Provenance Label: ● Verified ● Self-reported ● Needs Attention ● HR Override. Rows update themselves within 30 seconds of new watch data — no manual refresh. HR can manually mark someone ready independent of the automatic signal; this override is always kept visually and structurally distinct from machine-verified data, and can be reversed. Status badges never rely on color alone.

4.5 — Authentication & Login

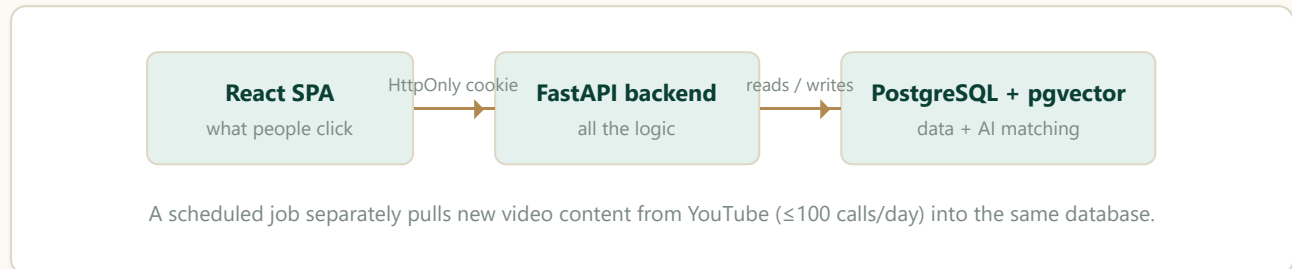
No assignment, content, or watch-progress data is reachable without logging in — never a flash of protected content before redirect. Each session is locked to exactly one role: an HR Admin sees org-wide (through safe, coaching-shaped views); an Employee only ever sees their own data, no matter how a request is phrased. Signing out kills the session immediately.

Quality bar (non-functional requirements)

CONCERN	WHAT IT MEANS
Speed	Dashboard loads <2s; video player <3s; new assignment appears <1s after confirming.
Data safety	A failed dashboard refresh never means a lost assignment — shown as a distinct, recoverable error.
Anti-cheating	Server rejects unrealistic jumps (e.g. instant leap to 100%) before trusting a write as Verified.
Privacy	Raw watch data can never feed a performance review — enforced in code, not just policy.
Accessibility	WCAG 2.1 AA — keyboard-operable, screen-reader announcements, no color-only signals.
Budget	Zero paid infrastructure; free/local tooling throughout.

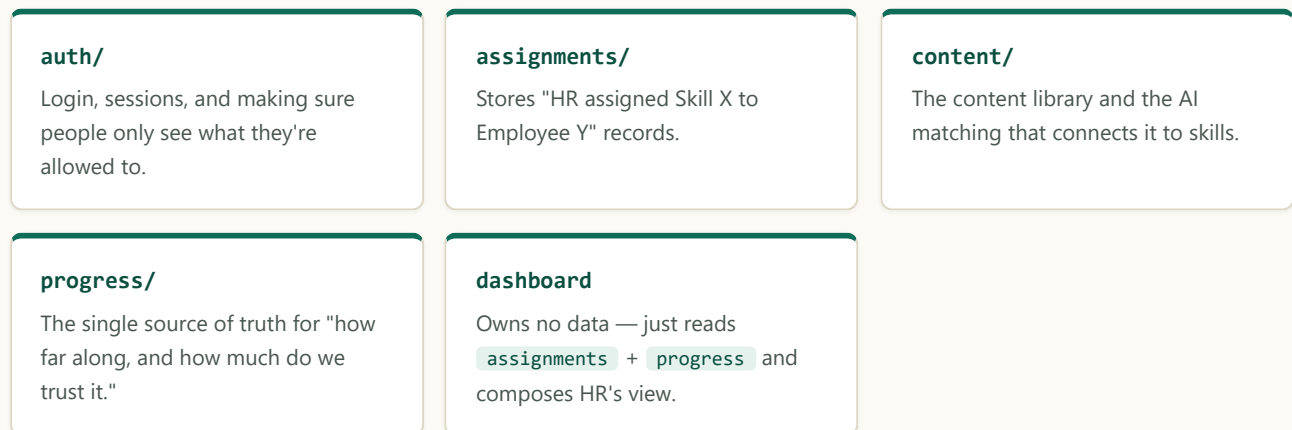
05 Architecture, in Plain Language

TalentPilot-AI is built as a **modular monolith** — one application, not a tangle of microservices, but internally organized into clean, self-contained sections so any one feature can be understood by opening a single folder. For a small internal pilot, microservices would be pure overhead; this keeps structure without the cost.



Everything runs locally via Docker for this pilot — there is no live production deployment yet. That's a deliberate decision to keep the pilot scoped and cheap.

The five building blocks



The most important design decision

Only one module (`progress/`) is allowed to decide what a row's Status and trust level actually are. No other part of the system computes or guesses this independently.

Why it matters: if the dashboard, the assignment screen, and some future export each calculated "is this person ready?" with their own logic, they could disagree — which is exactly the trust problem this product exists to solve. Funneling every read through one authority keeps the answer consistent everywhere, and makes the "coaching-only" privacy rule (watch data must never leak into a performance review) enforceable in exactly one place instead of scattered — and easily forgotten — across the code.

Other key decisions, explained

- **Ordering by time, not position.** A rewind should count even though the position number drops. The system tells "someone rewound on purpose" apart from "a stale update arrived late" by comparing timestamps, never raw positions.
- **An HR Override never overwrites real data.** It's stored as its own record, so if fresh watch data arrives later, both stay visible — nothing silently erased, and an override is never disguised as

machine-verified.

- **Content is fetched in batches, not searched live.** YouTube's search API allows ~100 calls a day — nowhere near enough for live per-page search — so the library is pre-built and matching runs against what's already collected.
- **Matching uses AI "embeddings," not exact keywords.** A free, offline model converts skill names and video descriptions into comparable vectors. If nothing is close enough, the system shows nothing rather than a bad guess.
- **The video player sits behind a swappable adapter.** The progress-tracking logic doesn't know or care whether the video is YouTube or (later) Vimeo.
- **Every request is checked server-side, every time** — not just at login. An employee session is hard-locked to that employee's own data no matter how a request is phrased.
- **The dashboard polls every ≤ 30 seconds** rather than using more complex real-time push technology — simple, reliable, and sufficient at this scale.

Tech stack, at a glance

LAYER	TOOL	WHY
Frontend	React + TypeScript + Vite, shadcn/ui + Tailwind	Modern, fast, well-supported
Backend	Python 3.12+, FastAPI	Fast to build with, strong async support
Database	PostgreSQL + pgvector	Relational storage plus AI similarity search, one system
AI matching	Local sentence-transformers model	No API cost, works offline, fits zero-budget
Video	YouTube IFrame Player, behind an adapter	Free, documented, not locked in long-term
Hosting	Docker Compose, local only (pilot)	Keeps the pilot cheap and simple

06 Open Questions & Risks

Documented honestly during planning — none block building the pilot, but they matter for what comes after.

-
- 01 **Where do accounts and the employee roster come from in production?** Seeded locally by hand for now; real SSO / HRIS integration is a later decision.
-
- 02 **How long should watch history be kept?** No retention period has been locked in yet.
-
- 03 **Do HR Admins actually notice and correctly read the trust labels?** Untested with a real HR person — flagged for a check right around launch, since the product's value depends on HR actually using this signal.
-
- 04 **Documents and websites stay self-reported indefinitely** in this version — only video can be automatically verified. Known, accepted limitation.
-
- 05 **No formal legal/compliance review** of employee video-watch tracking has happened — consciously deferred for an internal pilot, revisit if scope expands.
-
- 06 **No long-term owner named yet** for after the pilot. Three possible futures exist (stay standalone / feed a future LMS / fold into a bigger platform) — no decision made.
-

07 Source Documents

DOCUMENT	COVERS
brief.md	Original problem framing, opportunity, business case
prd.md	Full detailed requirements — the 14 FRs, user journeys, non-goals, success metrics
ARCHITECTURE-SPINE.md	The binding technical rules a builder must not violate
SOLUTION-DESIGN.md	Human-readable explanation of why those rules exist, plus data model and flows
epics.md	Requirements broken down into buildable epics and stories

All sourced from `_bmad-output/` in the project repository.